

ADIVINHE O NÚMERO

Arthur Silva de Albuquerque	–	17078155
Bruno Lopes de Almeida Zuffo	–	15444031
Cody Stefano Barham Setti	–	4856322
Matheus Valim Nogueira	–	15746323

21 de setembro de 2026

São Carlos, SP, Brasil

Principais Desafios & Soluções

- **Geração de números aleatórios sem instrução de tempo real confiável**

Optamos por usar `ecall 30` (tempo do sistema) apenas como semente inicial do LCG, garantindo que o resultado do algoritmo — e não o relógio em si — determinasse o número sorteado, conforme exigido pelo enunciado.

- **Overflow na multiplicação do LCG**

Como $a * X(n)$ pode ultrapassar os 32 bits, usamos a instrução `mul` (que retorna os 32 bits menos significativos do produto) e aplicamos uma máscara (and com `0x7FFFFFFF`) para simular o módulo por 2^{31} , evitando a necessidade de aritmética de 64 bits.

- **Alocação da lista ligada exclusivamente na heap**

O enunciado proíbe o uso da pilha para os dados da lista. Resolvemos isso chamando `sbrk (ecall 9)` a cada novo palpite, retornando um endereço “fresco” na heap para cada nó; a pilha (`sp`) foi usada somente para salvar/restaurar registradores nas chamadas de função (uso previsto pela convenção de chamada, não para os dados do jogo).

- **Manutenção da ordem cronológica das tentativas**

Para imprimir os palpites na ordem em que foram digitados, mantivemos um ponteiro de cauda (`tail`, registrador `s2`) além do de cabeça (`head`, registrador `s1`), permitindo inserção em $O(1)$ ao final da lista em vez de inversão da ordem.

Lições Aprendidas

- A implementação reforçou a importância de seguir rigorosamente a convenção de chamada do RISC-V (uso correto de registradores salvos vs. temporários), especialmente ao encadear várias chamadas de função dentro do laço principal do jogo.
- Trabalhar com estruturas de dados dinâmicas (lista ligada) em Assembly exige atenção redobrada ao gerenciamento manual de ponteiros, já que não há verificação automática de tipos ou de limites como em linguagens de alto nível.
- A separação do código em funções pequenas e bem definidas facilitou tanto a depuração no simulador RARS quanto a legibilidade e manutenção do código-fonte.